Vibration-Based Structural Damage Detection Using an ARM Cortex-M3 Microcontroller

Sagnik Basu
Reg. No. 240953528
Roll No. 42

Pranav Gaddam
Reg. No. 240953548
Roll No. 43

Shobit Hegde
Reg. No. 240953556
Roll No. 44

School of Computer Engineering
Section CCE A
Manipal Institute of Technology, Manipal, India

Abstract*— Structural health monitoring (SHM) can detect damage at an early stage. However, most vibration-based SHM systems require expensive data acquisition hardware and computer analysis. The most common damage indicator, natural frequency, is not always effective at detecting faults and will change due to benign factors such as extra mass. Most recent work on TinyML has focused on powerful processing units and there is a lack of fully verified systems using a simple microcontroller. A damage detection system is presented which runs entirely on the LPC1768 ARM Cortex-M3. The MPU6050 MEMS accelerometer samples free vibration of a steel cantilever at 100 Hz. A windowed 1024-point FFT with parabolic interpolation gives the natural frequency, and a log envelope fit gives damping. Both are taken relative to a baseline set on the board from ten healthy flicks, so the system adapts to a new board or setup without retraining. A decision tree, k-nearest neighbours, a 2-8-4 neural network, a health score and a change detector run on the board; all were trained offline and converted into C code. On 131 lab recordings over two days and two boards, covering a healthy state, extra mass, loose joints and loosened supports, kNN reached 96.0% accuracy and the neural network 93.1%, against 78.0% for a frequency-only rule. The health score had an average error of 4.3 points. A live browser dashboard shows the results of every flick. Explainable SHM can be implemented in about 20 KB of flash memory on an entry-level microcontroller.

***Index Terms*—ARM Cortex-M3, edge machine learning, MEMS accelerometer, structural health monitoring, TinyML, vibration-based damage detection**

I. Introduction

Civil and mechanical structures degrade over time due to cracks, corrosion and loose joints. The purpose of structural health monitoring (SHM) is to automatically detect this damage, ideally at an early stage, by identifying if damage exists, locating its position, determining the damage type and severity [1]. The main advantage of vibration-based techniques is that any variation in stiffness, mass or boundary conditions will affect the dynamic response and a single accelerometer can be used [2], [3].

A common metric used is the first natural frequency. Although inexpensive to calculate, there are two main disadvantages. Firstly, many faults cause very little change in this metric and also benign changes, such as added mass or temperature, can have a similar effect to real damage [2]. By using machine learning (ML), multiple features can be combined and the patterns within each condition can be learnt [3], [4], [5]. Most ML-based SHM systems require a PC/cloud system to train and operate, requiring a constant stream of data along with dedicated data acquisition hardware [6], [7].

TinyML takes the inference stage of ML to run on a microcontroller close to the sensor [8]. This has advantages of less data transmission, lower power and cost. However, current literature surrounding edge-based SHM uses multi-core or DSP capable chips and complex neural networks. This project aims to reduce complexity by asking if it is possible to run a fully explainable damage detection pipeline, taking raw acceleration data and producing named faults and a health rating, on an entry-level ARM Cortex-M3 chip without a floating point unit, within 32 KB of code space. The results will also be validated using real hardware.

The contributions of this paper are:

1) An assemble-only cantilever test bench made from two steel rulers, binder clips and a screw clamp, on which three fault types are introduced repeatably and without tools.
2) Fixed-point-friendly firmware for the LPC1768 that samples an MPU6050 at 100 Hz and extracts five features, including interpolated frequency and log-envelope damping, with a self-written radix-2 FFT.
3) Six lightweight models (decision tree, kNN, 5-8-4 neural network, kNN health regression, z-score change detector and novelty detector) trained on a laptop and exported automatically as C code.
4) An experimental study on 131 recordings over two days and two boards showing that calibration-relative features keep the models accurate across boards, that the ML outperforms a frequency-only rule, and that board and PC outputs are identical.

II. Literature Review

A. Vibration-based damage identification

Hou and Xia [2] reviewed damage detection methodologies using vibrational responses from civil engineering structures. The review classifies methodologies according to the vibration features used (natural frequencies, mode shapes, damping and frequency response functions), with frequency being the simplest to measure but providing little information on the nature or position of damage. Gharehbaghi et al. [1] provide a

formal definition of the levels within SHM and compare model-based to data-driven methodologies. Both highlight the issues with environmental and operational variability that can disguise or mimic damage. This highlights why an added mass test was performed for this work.

B. Machine learning for SHM

The transition from manually set thresholds to ML-based solutions using deep learning has been described [3]. A feature extraction stage with a classifier is the most popular form of structure. A comprehensive review of supervised and unsupervised algorithms in civil SHM was carried out [4]. The main issue with implementing these methods is that the available data sets are small. ML provides a novel approach to SHM which can be applied to the identification of damage, detection of anomalies in data and provide an assessment of the condition [5]. A review of ML within SHM was completed alongside emerging sensing technologies [6], such as low-cost MEMS sensors and IoT nodes. A study which is similar to this study was conducted using kNN, support vector machines, decision trees, random forests and naive Bayes algorithms to classify damage in cantilever beams [10]. When only natural frequencies were considered as features the accuracy of the models varied from about 59% to 94%. However, using more complex features from the response the accuracy improved to around 95% using the experimental data.

C. Loosening as a fault

Loose supports and loose bolted joints are among the most frequent problems in real structures. Gong et al. review the research on loosening of threaded fasteners and show that it is driven by slippage and friction at the thread and bearing contact surfaces [11]. A loosening joint therefore loses clamping stiffness and dissipates energy through friction at the contact interface, which shows up as changes in frequency and damping. This is why damping has been selected as a feature.

D. Frequency estimation

The frequency can only be determined up to $\Delta f = F_s/N$ because the record has a finite length N. A much more accurate estimation can be obtained by fitting a parabola to the three highest samples of the DTFT around the peak of interest. This has minimal additional computational complexity [12] and is applied on the board.

E. SHM at the edge and TinyML

Zonzini *et al.* [7] propose a sensor-to-cloud SHM architecture in which MEMS sensor nodes are used to pre-process data before sending it. Ray [8] and Capogrosso *et al.* [9] review TinyML frameworks, model compression and deployment on microcontrollers. Specific to SHM, Burrello *et al.* [13] train tiny autoencoders to detect anomalies near the sensor, Moallemi *et al.* [14] accelerate output-only system identification on a parallel RISC-V microcontroller, and Capello *et al.* [15] use neural architecture search to fit SHM models on resource-constrained devices.

F. Research gap

Table I shows a summary of the most closely related work. Previous edge-based SHM work has used powerful processors and either anomaly-only or black-box models. The beam classification work has been done using a PC. No previous work has classified fault types, given severity scores and implemented a change detection algorithm using a simple Cortex-M3 processor. This work has been tested using real data with identical outputs from the PC and the board.

TABLE I
COMPARISON WITH CLOSELY RELATED WORK

Work	Platform	Output	Features	Data
[10]	PC	fault class	freq., FRF	sim. + exp.
[13]	MCU (multi-core)	anomaly	raw signal	real
[14]	RISC-V MCU	modal param.	AR/ARMA	real
[15]	MCU	anomaly	learned	real
This work	Cortex-M3 (no FPU)	class, health, change, novelty	freq., damping (relative)	real (131 flicks)

III. METHODOLOGY

A. Overall approach

The methodology used is shown in Fig. 1. Each labelled flick is stored on the board and then transferred to the laptop via UART. The laptop extracts the same features as the board, expresses them relative to the healthy baseline of that session, and trains and cross-validates the models. Once trained, they are exported into a C header file (model.h) which is then compiled into the firmware. In use, the board first records ten healthy flicks (key C) to set its own baseline, and all new flicks are then classified on the board. Finally, a parity check runs the same C code on the PC over every recording and compares its output with Python.

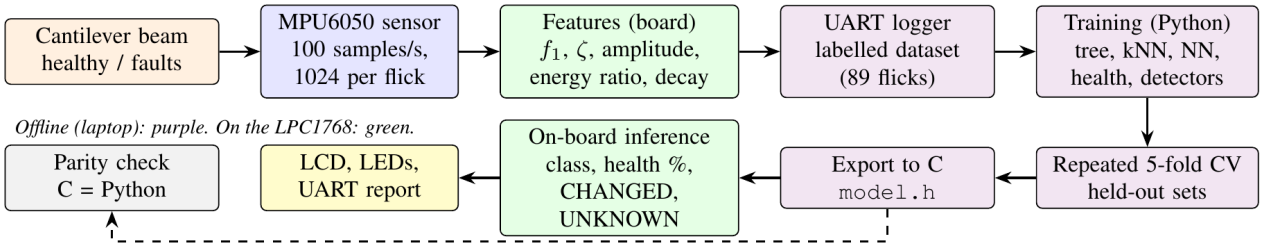


Fig. 1. Method approach: data collection, offline training and on-board deployment.

B. Test structure and fault cases

The structure is a cantilever made out of two 30 cm steel rulers (Fig. 2). Ruler A projects 18.5 cm beyond the table edge and is held by a 10-inch screw G-clamp over a cardboard pad. Ruler B overlaps A by 13.5 cm and is joined to it by three binder clips, giving a free length of about 35.5 cm. The MPU6050 module is taped near the tip. Each measurement starts with a manual flick of the tip, after which the beam rings freely. The build needs no drilling or cutting, so the bench can be reassembled in any lab.

Three fault types were introduced (Fig. 3). *Added mass*: a Rs 20 and a Rs 5 coin taped together at the 28-30 cm mark. This is a change but not damage. *Loose joint*: the middle binder clip removed (level 1), then one end clip removed as well (level 2). *Loose support*: the G-clamp screw loosened in three steps of about 1/4, 1/2 and 3/4 turn; 3/4 turn is the loosest setting at which the beam still stays in place.

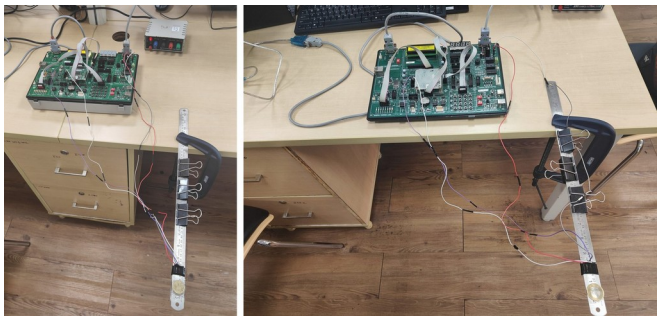


Fig. 2. Experimental setup. Left: LPC1768 board, power supply and the clamped two-ruler cantilever with the MPU6050 near the tip. Right: side view showing the three binder clips, the G-clamp and the wiring to the board.

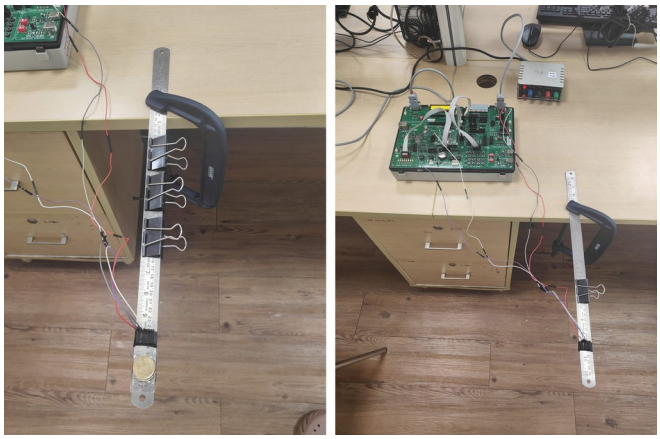


Fig. 3. Fault cases. Left: added mass, coins taped beyond the sensor. Right: loose joint, clips removed from the ruler overlap.

C. Hardware

Fig. 4 shows the block diagram. The LPC1768 (ARM Cortex-M3, 100 MHz, 512 KB flash, 64 KB SRAM) sits on the ALS-SDA-ARMCTXM3-01 development board. The MPU6050 accelerometer is read over a software I²C bus on two GPIO pins, with its measuring range set to ± 4 g, so it can read accelerations of up to four times gravity in either direction, and its internal low-pass filter set to 44 Hz to prevent aliasing. Timer 0 triggers a reading every 10 ms (100 samples per second). The board's 16×2 LCD shows the result. Eight LEDs form a health bar. The 4×4 keypad selects calibration, teach mode and display pages. UART0 at 19200 baud sends every result and, while logging, the raw samples to the laptop.

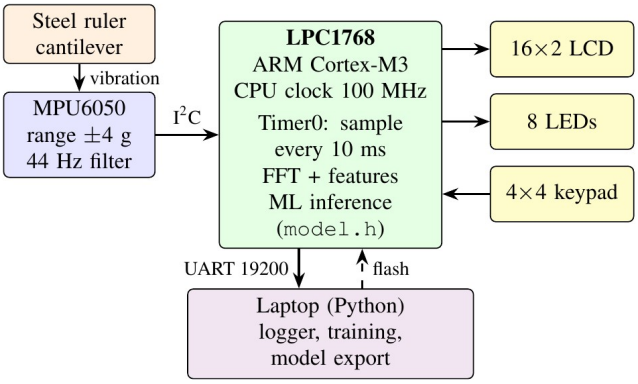


Fig. 4. Block diagram of the hardware.

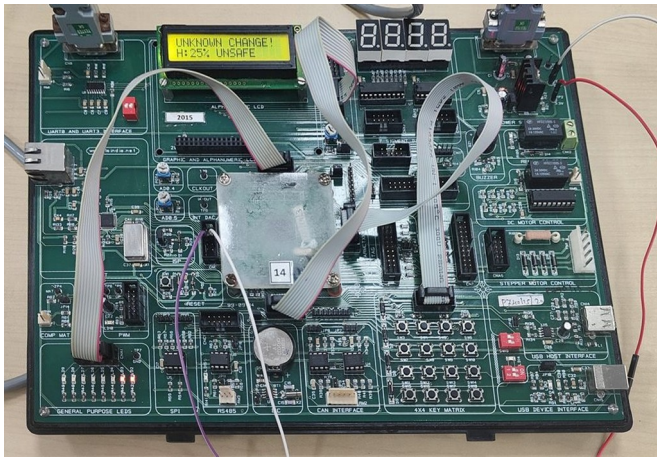


Fig. 5. LPC1768 board during a fault test. The LCD shows the change alarm and the health score.

D. Signal processing and features

A flick is detected when the acceleration moves more than 0.3 g away from its resting value. The next $N = 1024$ samples are then recorded at $F_s = 100$ Hz, which takes 10.24 s. Physically, the method can be understood using single degree of freedom theory for the first mode of the beam,

$$f = \frac{1}{2\pi}\sqrt{\frac{k}{m}}, \tag{1}$$

so a loss of stiffness k, or a gain in mass m lowers the natural frequency [2]. After removing the mean, the record is multiplied by a Hann window to reduce spectral leakage,

$$w[n] = 0.5 - 0.5\cos\left(\frac{2\pi n}{N-1}\right), \tag{2}$$

and transformed with a radix-2 FFT,

$$X[k] = \sum_{n=0}^{N-1} x[n]\,w[n]\,e^{-j2\pi kn/N}. \tag{3}$$

The bin width is $\Delta f = F_s/N = 0.098$ Hz, and the largest bin k_p between 3 and 40 Hz is refined with parabolic interpolation through its magnitude β and its neighbours α and γ [12]:

$$\delta = \frac{1}{2}\frac{\alpha-\gamma}{\alpha-2\beta+\gamma}, \quad f_1 = (k_p+\delta)\,\Delta f . \tag{4}$$

Damping is taken from the free decay. For a lightly damped mode the amplitude decays as $A(t) = A_0 e^{-\zeta\omega_1 t}$ with $\omega_1 = 2\pi f_1$. Taking logarithms gives a straight line, so the damping ratio is

$$\zeta = \frac{-s}{2\pi f_1}, \tag{5}$$

where s is the least-squares slope of $\ln$ RMS against time, computed over 0.25 s windows from the loudest window until the RMS falls below 5% of its peak. Three further features are the peak amplitude a_{max} (flick strength), the energy ratio

$$E_r = \frac{\sum_{k\in B}|X[k]|^2 - \sum_{|k-k_p|\le h}|X[k]|^2}{\sum_{k\in B}|X[k]|^2}, \tag{6}$$

which measures energy outside ± 1 Hz of the main peak (rattle and harmonics) within the band B of 3–40 Hz, and the decay time t_d until the RMS drops below 10% of its peak. The feature vector is $x = [f_1, \zeta, a_{max}, E_r, t_d]$.

E. Models

All models use two calibration-relative features: the ratio of f1 to the baseline f1 and the change in damping from the baseline damping, both standardised, $\tilde{x}_i = (x_i - \mu_i)/\sigma_i$.

Frequency-only baseline: a depth-2 tree on f_1, representing the classical frequency-shift rule.

Decision tree: CART of depth 3 with Gini impurity $G = 1 - \sum_c p_c^2$; it is exported as nested `if` statements.

k-nearest neighbours: $k = 3$ with Euclidean distance over the 101 stored training flicks; new examples can be added on the board in teach mode.

Neural network: a 2-8-4 multilayer perceptron,

$$h = max(0, W_1\tilde{x} + b_1), \quad p = softmax(W_2 h + b_2), \tag{7}$$

with 60 parameters; the largest p_c is the predicted class and its value the confidence.

Health score: each training condition has a damage level $d \in [0,1]$ (healthy 0, added mass 0.15, loose joint 0.5 and 1.0, loose support up to 0.75). The health of a newly done flick is

$$H = 100\left(1 - \frac{1}{3}\sum_{j=1}^{3} d_j\right), \tag{8}$$

where d_j are the levels of its three nearest neighbours. $H < 40$ is reported as UNSAFE.

Change detector: the distance from the healthy baseline,

$$z = \sqrt{\frac{1}{5}\sum_{i=1}^{5}\left(\frac{x_i - \mu_i^h}{\sigma_i^h}\right)^2}, \tag{9}$$

flags CHANGED when $z > 3$. The baseline μ^h, σ^h is set on the board from ten healthy flicks (key C); the median of these flicks is used, so one bad flick does not move it. Key B lets the operator mark the support as loose (health capped at 50%) and key D clears the mark. *Novelty detector*: UNKNOWN is shown when the nearest stored flick is farther than twice the 95th percentile of nearest-neighbour distances in the training set.

Trend: the health of successive inspections is fitted with a line $H = a + bt$, and the step at which H reaches 40 is predicted as $(40 - a)/b$.

F. Firmware flow

Fig. 6 shows the firmware loop. All models are compiled from the exported header, so no training happens on the board.

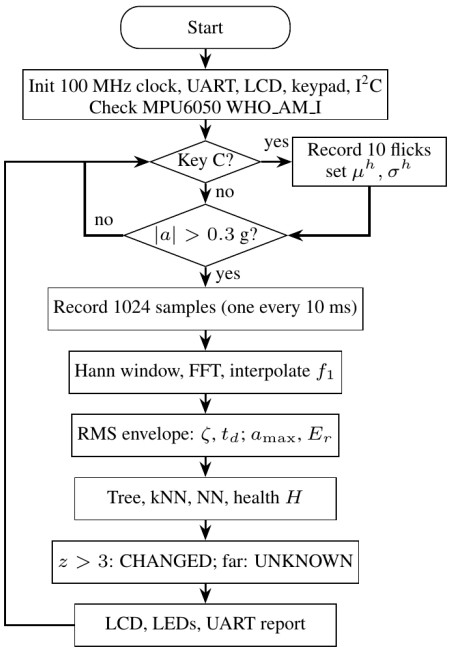


Fig. 6. Flowchart of the on-board firmware.

G. Data collection and evaluation

Each flick is labelled on the laptop and stored with the raw sample data. Data was recorded on two days (Table II): day 1 on the original board and day 2 on a different LPC1768 board, on which the same healthy beam read about 4.4 Hz instead of 4.74 Hz. The cause of this offset was not found, which motivated the switch to features relative to a healthy baseline taken at the start of each session. Prior to each fault, a check was performed on the beam configuration to ensure it was identical to that of a healthy device. Any flicks that resulted in

an incorrect overlap of 15 cm between the two rulers were discarded (24 in total). The first two steps of support loosening were excluded from training as their features were within the healthy range (Section IV-C), and were used as a held-out test with five healthy verification flicks. The accuracy is calculated by using 5-fold stratified cross-validation repeated 5 times on the 101 training flicks.

TABLE II
RECORDED DATASET, DAY 1 / DAY 2 (MEDIANS)

Condition	Flicks (day 1 / 2)	f_1 (Hz)	ζ (%)	Use
Healthy	30 / 7	4.74 / 4.39	2.92 / 2.53	train
Added mass	12 / 5	3.68 / 3.35	2.58 / 1.65	train
Loose joint 1	10 / 11	4.47 / 4.25	2.67 / 1.91	train
Loose joint 2	8 / 5	4.45 / 4.02	1.16 / 1.00	train
Loose support 3/4 turn	8 / 5	4.75 / 4.71	3.45 / 3.11	train
Loose support 1/4 turn	8 / 5	4.71 / 4.43	3.26 / 3.14	held out
Loose support 1/2 turn	8 / 4	4.75 / 4.46	3.13 / 2.78	held out
Healthy re-check	5 / 0	4.68 / -	2.95 / -	held out
Total	89 / 42 = 131			

IV. RESULTS AND DISCUSSION

A. Frequency estimation on the board

Table III compares three frequency estimators against a 16 × zero-padded FFT reference over the 89 day-1 flicks, with execution times measured on the LPC1768 using SysTick. Parabolic interpolation reduced the mean error from 0.027 Hz to 0.009 Hz, less than a tenth of the bin width, at no measurable extra time. Zero-crossing counting is 28 times faster but its mean error of 0.34 Hz is larger than the 0.27 Hz shift caused by a loose joint, so it cannot separate the classes (Fig. 7). The FFT takes 117 ms, which is negligible against the 10.24 s recording.

TABLE III
FREQUENCY ESTIMATORS MEASURED ON THE LPC1768

Method	Mean err.	Max err.	Time	RAM
FFT peak bin	0.027 Hz	0.052 Hz	116.9 ms	8 KB
Interpolated FFT, (4)	**0.009 Hz**	0.063 Hz	116.9 ms	8 KB
Zero-crossing	0.338 Hz	2.07 Hz	4.2 ms	none

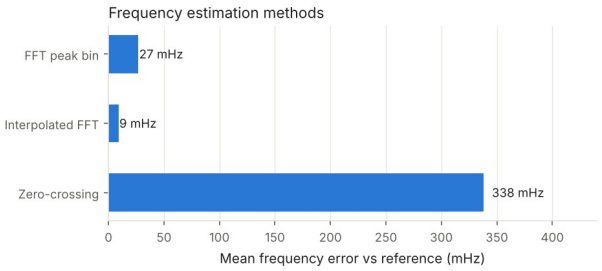


Fig. 7. Error and execution time of the three frequency estimators.

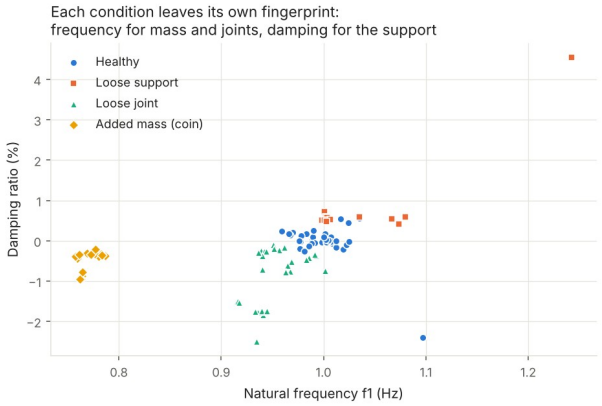


Fig. 8. Damping against first natural frequency for all training flicks.

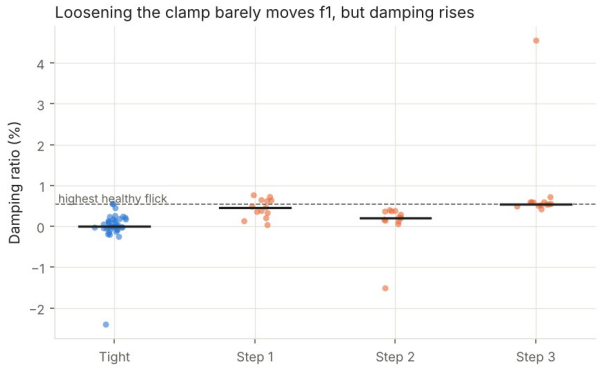


Fig. 9. Damping ratio at each support-loosening step compared with the healthy range.

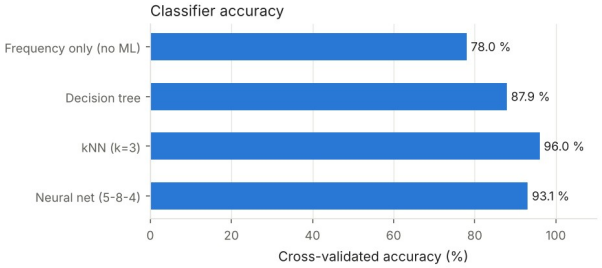


Fig. 10. Accuracy of the frequency-only rule and the three classifiers.

B. Each fault has its own signature

Fig. 8 shows the relative damping and frequency of each flick. The added mass reduced f1 by about 22% on both days, as predicted by (1). Removing clips decreased f1 by around

5% to 8% due to a loss of stiffness in the joint. Removing the second clip also decreased damping from about 2.5% to 1.0%, because less friction remains at the overlap. When the support is loosened f1 hardly changes, but damping increases because the loose end loses energy through friction. This matches the loosening literature [11]. This fault cannot be seen using frequency alone. Although the absolute frequencies differ by about 0.35 Hz between the two boards, the relative changes for each fault are the same, so one model serves both.

C. Detection limit for support loosening

Fig. 9 shows damping at each loosening step. At 1/4 and 1/2 turn the damping stays close to the healthy range, and only at 3/4 turn is it clearly higher. The screw clamp maintained a tight grip on the ruler until it was almost released, so the first two steps were near or below the detection threshold of this system. On these held-out flicks, the neural network labelled 7 of 13 (1/4 turn) and 0 of 12 (1/2 turn) as loose support, consistent with the features lying on the edge of the normal cluster. For this reason the firmware also offers key B, so an operator who sees a loose clamp can mark it directly.

D. Classification accuracy

Table IV and Fig. 10 give the cross-validated accuracy. All three ML models beat the frequency-only rule (78.0%), by 10 to 18 percentage points, and kNN is best at 96.0%. The decision tree uses f_1 for 43% of its splits and damping for 57%, confirming that the gain comes from the damping feature. The confusion matrices in Fig. 11 show that most remaining errors are between healthy and loose support, the hardest pair. All five healthy re-check flicks were classified as healthy with no CHANGED alarm. Using all five features (adding amplitude, energy ratio and decay time) was also tested: accuracy fell to 85.7% (kNN), 87.3% (neural network) and 88.6% (tree), and the health error rose to 10.1 points, because these extra features vary with the strength of the hand flick and between boards. The final model therefore uses only the two relative features.

TABLE IV
CROSS-VALIDATED CLASSIFICATION ACCURACY (4 CLASSES)

Model	Accuracy
Frequency only (depth-2 tree on f_1)	78.0%
Decision tree, depth 3	87.9%
kNN, $k=3$	96.0%
Neural network 2-8-4	**93.1%**

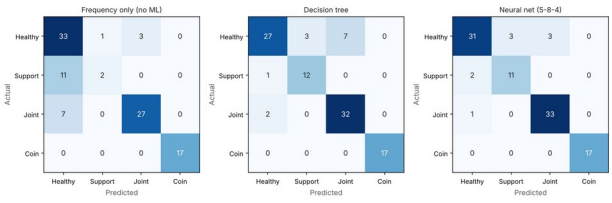


Fig. 11. Confusion matrices from cross-validation (left to right: frequency only, decision tree, neural network).

E. Health score and trend

The kNN health score had a mean absolute error of 4.3 points, against 29.1 points for linear regression on the same features (Fig. 12). With the added-mass damage level set to 0.15, a coin reads about 85% rather than being treated as damage, while the loose joint reads about 50% at level 1 and close to 0% at level 2, which is reported as UNSAFE. On the day-1 support-loosening steps, a line fitted to steps 0 to 2 predicted the beam would become unsafe at step 3.6; in the experiment it was unsafe at step 3, so the forecast was slightly late but in the right range (Fig. 12, right).

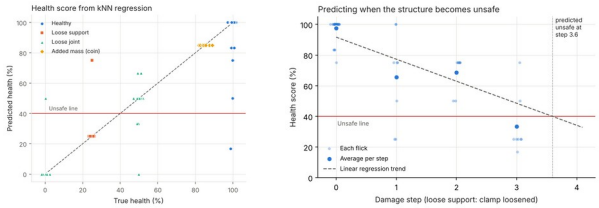


Fig. 12. Left: predicted against true health score. Right: health over successive loosening steps and the predicted unsafe point.

F. On-board operation and reporting

The full firmware, including the FFT, all models, and training data for the kNN algorithm requires approximately 20 KB of flash memory to store. This is well within the 32 KB limitation of the free Keil toolchain. The FFT buffers also require 8 KB of RAM. The C code was run through all flicks on the PC and produced the same features and classifications as the Python reference implementation. Fig. 5 shows the board detecting a change and an unsafe health value during a fault test. For each flick the board also sends its result and raw samples over UART to a laptop, where a browser dashboard (Fig. 13) shows the health gauge, the class chosen by each model, the waveform with its decay envelope, the spectrum, the calibration values and a health timeline. It runs offline and needs no internet.

Finally, the laptop transmits the results for each condition (median frequency, damping, health status, predicted class, confidence, cross-validated accuracies and trend prediction) to a large language model (gpt-oss-120b using the Groq API and using Gemini as a fallback). The language model is then prompted to produce a brief inspection summary in simple English. The language model does not make any decisions; all numbers come directly from the measured table, and all detection and classification is performed by the models on the board. In our case, the report correctly identified loose joint 2 and the 3/4 turn support as being unsafe, and identified the added mass as being harmless. However, the confidence of the classifier was exaggerated for the first two steps of loosening. Therefore, the report should always be checked against the table.



Fig. 13. Live browser dashboard during a loose-joint test: health gauge, model votes, modal features, waveform, spectrum, health timeline and calibration values.

G. Contributions verified through results

Each contribution is backed by a measured result. The assemble-only bench allowed repeatable results: the day-1 healthy re-check after all faults read 4.68 Hz against the original 4.74 Hz, and all 5 healthy checks were found to be healthy. Using the interpolated FFT algorithm on the Cortex-M3 gave an average frequency error of 0.009 Hz, less than a tenth of the 0.098 Hz bin width and three times better than the peak bin alone (0.027 Hz), in 117 ms. Adding damping to frequency, both relative to an on-board baseline, improved 4-class accuracy from 78.0% (classical frequency method) to 96.0% (kNN) across two days and two boards. The kNN model predicted the severity of damage with an average error of 4.3 points. Exporting the models to C kept the same features and logic as Python, giving identical results, and everything fits in about 20 KB of flash memory. An honest upper limit was also found: a support loosening of less than about 3/4 turn could not be reliably told apart from a healthy state using this sensor and structure.

H. Limitations

The dataset is still small (131 flicks) and comes from one structure in one room, so temperature and long-term drift were not studied. The reason the second board read a lower frequency for the same beam was not found; calibration hides the offset but does not explain it. Flicks were applied by hand, which varies the amplitude; a mechanical exciter would give cleaner data. Only the first mode was used.

V. Conclusion

This paper demonstrates that a full damage detection system using vibration data, producing a fault name, health value and trend prediction from raw acceleration data, can be implemented on an entry-level ARM Cortex-M3 processor and a low-cost MEMS accelerometer. On 131 recordings over two days and two boards, the tool-free cantilever test rig gave an accuracy of 96.0% (kNN) in distinguishing between healthy, added mass, loose joint and loose support cases, compared to 78.0% using the classical frequency shift rule. The health value had an average error of 4.3 points. Calibrating on ten healthy flicks made the same model work on a board that read a different absolute frequency, and damping, which is not considered by the frequency rule, was the main source of information regarding loose supports. All models fit within approximately 20 KB of flash memory and behave identically to the PC version, and a live dashboard makes every result visible. In the future the system will be improved by implementing a mechanical exciter, temperature compensation, higher modes from a second sensor and more data to reduce the detection limit for early loose support.

Acknowledgment

The authors thank their guides, Dr. G Pradeep Reddy and Pragya Jha, Assistant Professors, School of Computer Engineering, Manipal Institute of Technology, for their guidance, and the Embedded Systems Laboratory for the LPC1768 boards.

References

[1] V. R. Gharehbaghi, E. Noroozinejad Farsangi, M. Noori, T. Y. Yang, S. Li, A. Nguyen, C. Málaga-Chuquitaype, P. Gardoni and S. Mirjalili, “A critical review on structural health monitoring: Definitions, methods, and perspectives,” *Archives of Computational Methods in Engineering*, vol. 29, no. 4, pp. 2209–2235, 2022, doi: 10.1007/s11831-021-09665-9.

[2] R. Hou and Y. Xia, “Review on the new development of vibration-based damage identification for civil engineering structures: 2010–2019,” *Journal of Sound and Vibration*, vol. 491, Art. no. 115741, 2021, doi: 10.1016/j.jsv.2020.115741.

[3] O. Avci, O. Abdeljaber, S. Kiranyaz, M. Hussein, M. Gabbouj and D. J. Inman, “A review of vibration-based damage detection in civil structures: From traditional methods to machine learning and deep learning applications,” *Mechanical Systems and Signal Processing*, vol. 147, Art. no. 107077, 2021, doi: 10.1016/j.ymssp.2020.107077.

[4] M. Flah, I. Nunez, W. Ben Chaabene and M. L. Nehdi, “Machine learning algorithms in civil structural health monitoring: A systematic review,” *Archives of Computational Methods in Engineering*, vol. 28, no. 4, pp. 2621–2643, 2021, doi: 10.1007/s11831-020-09471-9.

[5] Y. Bao and H. Li, “Machine learning paradigm for structural health monitoring,” *Structural Health Monitoring*, vol. 20, no. 4, pp. 1353–1372, 2021, doi: 10.1177/1475921720972416.

[6] A. Malekloo, E. Ozer, M. AlHamaydeh and M. Girolami, “Machine learning and structural health monitoring overview with emerging technology and high-dimensional data source highlights,” *Structural Health Monitoring*, vol. 21, no. 4, pp. 1906–1955, 2022, doi: 10.1177/14759217211036880.

[7] F. Zonzini, C. Aguzzi, L. Gigli, L. Sciullo, N. Testoni, L. De Marchi, M. Di Felice, T. S. Cinotti, C. Mennuti and A. Marzani, “Structural health monitoring and prognostic of industrial plants and civil structures: A sensor to cloud architecture,” *IEEE Instrumentation & Measurement Magazine*, vol. 23, no. 9, pp. 21–27, 2020, doi: 10.1109/MIM.2020.9289069.

[8] P. P. Ray, “A review on TinyML: State-of-the-art and prospects,” *Journal of King Saud University - Computer and Information Sciences*, vol. 34, no. 4, pp. 1595–1623, 2022, doi: 10.1016/j.jksuci.2021.11.019.

[9] L. Capogrosso, F. Cunico, D. S. Cheng, F. Fummi and M. Cristani, “A machine learning-oriented survey on tiny machine learning,” *IEEE Access*, vol. 12, pp. 23406–23426, 2024, doi: 10.1109/ACCESS.2024.3365349.

[10] A. A. S. R. de Sousa, J. S. Coelho, M. R. Machado and M. Dutkiewicz, “Multiclass supervised machine learning algorithms applied to damage and assessment using beam dynamic response,” *Journal of Vibration Engineering & Technologies*, vol. 11, no. 6, pp. 2709–2731, 2023, doi: 10.1007/s42417-023-01072-7.

[11] H. Gong, X. Ding, J. Liu and H. Feng, “Review of research on loosening of threaded fasteners,” *Friction*, vol. 10, no. 3, pp. 335–359, 2022, doi: 10.1007/s40544-021-0497-1.

[12] D. Belega and D. Petri, “Unbiased sine-wave frequency estimation by parabolic interpolation of DTFT samples,” in *Proc. IEEE Int.*

Instrumentation and Measurement Technology Conf. (I2MTC), 2021, pp. 1–5, doi: 10.1109/I2MTC50364.2021.9459910.

[13] A. Burrello, G. Sintoni, D. Brunelli and L. Benini, “Adversarially-trained tiny autoencoders for near-sensor continuous structural health monitoring,” in *Proc. IEEE 4th Int. Conf. Artificial Intelligence Circuits and Systems (AICAS)*, 2022, pp. 439–442, doi: 10.1109/AICAS54282.2022.9869952.

[14] A. Moallemi, R. Gaspari, F. Zonzini, L. De Marchi, D. Brunelli and L. Benini, “Speeding up system identification algorithms on a parallel RISC-V MCU for fast near-sensor vibration diagnostic,” *IEEE Sensors Letters*, vol. 7, no. 9, pp. 1–4, 2023, doi: 10.1109/LSENS.2023.3303074.

[15] A. Capello, R. Berta, M. Fresta, L. Lazzaroni and F. Bellotti, “Leveraging neural architecture search for structural health monitoring on resource-constrained devices,” in *Applications in Electronics Pervading Industry, Environment and Society*, Lecture Notes in Electrical Engineering. Cham, Switzerland: Springer, 2025, pp. 323–330, doi: 10.1007/978-3-031-84100-2_38.